不具合調査・改修設計仕様書: プロンプト上書きペースト時のステート非同期・消失不具合

～ TagChipEditor・PromptSectionClipboardMenu・Zustandストア間の同期是正 ～

不具合管理ID	BUG-PS-082 (PBI候補)	作成日	2026-09-27
種別	Bug Fix / State Synchronization	優先度 / 工数	P2 (Medium-High) / 0.5d
対象システム	Prompt Surgeon (comfy-create-helper2)	スタック	React 18 + Zustand + @dnd-kit
対象ファイル	DesktopLayout.tsx, MobileLayout.tsx, TagChipEditor/index.tsx	検証基準	tsc -b & Vitest (816+ passed)

1. エグゼクティブサマリー & 現象の整理

【報告された現象】
Generate画面のプロンプト入力欄付属「クリップボード・履歴操作メニュー（PromptSectionClipboardMenu）」から、ペーストモードを「上書き」に設定してプロンプトの貼り付けを実行した場合、入力欄のチップ表示（見た目）は即座に反映される。しかし、一度「テキスト入力モード切替」を行ってチップモードに戻す操作を挟まない限り内部で適用されておらず、ブラウザのリロードやタブ切り替え（セット切替・モバイルプロンプトタブ切替）を行うと、ペースト前の旧プロンプトに巻き戻ってしまう。

【調査結果と根本原因の結論】
調査の結果、本不具合は「文字列ステート（paramValues）とチップ配列ステート（promptChips）の更新非対称性」、および「TagChipEditor 内部の無限ループ抑止ガードによる親への同期通知遮断」の2つの要因が複合して発生していることが判明した。

■ 根本原因のポイント
① 親側の更新漏れ: クリップボードメニューのペースト発火時、親（DesktopLayout/MobileLayout）は setParamValues() のみを呼び出し、Zustandストアの promptChips を更新していない。
② 子の通知ガード: TagChipEditor は value 変更を検知して内部チップを更新するが、isExternalUpdateRef により onChipsChange() の発火を完全に抑制しているため、ストアの promptChips が古いまま残留する。
③ 再マウント時の巻き戻り: タブ切り替え時にコンポーネントが再マウントされ、古い promptChips が渡されて初期化されるため、ペースト前の状態に復元されてしまう。

2. 現象発生メカニズムとステート同期フローの詳細分析

Prompt Surgeon におけるプロンプト管理は、文字列としての `paramValues`（FastAPI/ComfyUI API送信用）と、ドラッグ＆ドロップやタグ単位の有効/無効トグルを司る `promptChips`（UI操作用）の2系統で同期保持されている。上書きペースト実行時のデータフローを以下に分解・分析する。

2.1 上書きペースト時の実行シーケンス

1. ペースト実行: `PromptSectionClipboardMenu.tsx` 内の `handleApplyPaste()` が実行され、`onChangeValue(cleanToPaste)` が発火する。
2. 親レイアウトの更新: `DesktopLayout.tsx` のハンドラ `onChangeValue={(val) => setParamValues({ ...paramValues, [role]: val })}` が実行される。この時、setPromptChips() は一切呼び出されない。
3. Zustandストアの状態: `paramValues[role]` は新しい文字列へ更新されるが、`promptChips[role]` および `activeSet.promptChips[role]` はペースト前の古い配列のまま維持される。
4. TagChipEditor への伝播: `TagChipEditor` の `value` プロップが更新されるが、`chips` プロップ（`chipsProp`）は古い参照のまま据え置かれる。

2.2 TagChipEditor 内部の同期処理と通知ガードの動作

`TagChipEditor/index.tsx` の外部同期 `useEffect`（Lines 95-108）において、以下の分岐が通過する：

```
// TagChipEditor/index.tsx (Lines 95-108)
} else if (value && value !== prevChipsString.current && value !== chipsToString(state)) {
    // chipsProp の参照変化はないが、value (文字列) が外部から変更された場合 (セクションコピペ等)
    isExternalUpdateRef.current = true;
    const newChips = parseToChips(value);
    setChips(newChips, false); // 内部ステート(state)を更新 → 見た目はチップで反映される
    prevChipsString.current = value;
    if (isTextMode) { setTextValue(value); }
}
```

ここで `setChips(newChips, false)` が呼ばれるため、コンポーネント内部のチップ表示は更新される（これが「見た目はチップで反映される」理由である）。
しかし、その直後に発火する親通知用の `useEffect`（Lines 125-144）において、致命的なブロックが発生する：

```
// TagChipEditor/index.tsx (Lines 125-144)
useEffect(() => {
    if (isInitialMountRef.current) { isInitialMountRef.current = false; return; }
    if (isExternalUpdateRef.current) {
        isExternalUpdateRef.current = false; // ★外部更新フラグを消費して即座に終了！
        return; // ★onChipsChange(state) が呼ばれず、親ストアへ一切通知されない！
    }
    if (!userEditedRef.current) { return; }
    userEditedRef.current = false;
    onChipsChange?.(state);
    ...
}, [state, onChipsChange, onChange, value]);
```

2.3 なぜ「一度テキストモードにする」と適用されるのか

ユーザーが「テキスト入力モード切替」をクリックし、再度「チップモード切替」を押下した場合、エディタ内の `handleSwitchToChipMode` が実行される。この関数内では以下のように処理される：

- `userEditedRef.current = true;` が明示的にセットされる。
- `isExternalUpdateRef.current = false;` の状態である。

このため、上記の親通知 `useEffect` 内のガード条件を通過し、`onChipsChange?.(state)` が確実に発火する。親の `handleChipsChange` を経由して `setPromptChips` が実行され、Zustandストア内の `promptChips` および `activeSet.promptChips` が最新のチップ配列で上書き保存されるため、これ以降は復元可能となる。

2.4 タブ切り替え・リロード・画像生成時の巻き戻り要因

テキストモードを経由しない場合、ストアの `promptChips` が古いまま残留するため、以下の3つの操作で破綻する：
1. タブ切り替え時（セット切替 / モバイルプロンプトタブ切替）：
`key={`${activeSetId}-${role}`}` の変更により `TagChipEditor` がアンマウントされ、再マウントされる。 初回レンダリング時に `initialChips` は `chipsProp`（古い配列）を優先して読み込み、 さらに `prevChipsString.current = chipsToString(chipsProp)` を設定するため、ペースト前の状態へ完全に巻き戻る。
2. 画像生成時（ComfyUI 投入プロンプトの不整合）：
`useGenerateAction.ts` 内の `getEffectiveParamValues(currentParamValues, currentPromptChips)` は、`currentPromptChips` に要素が存在する場合、文字列 `currentParamValues` よりも `promptChips` を優先して ComfyUI ワークフローへ流し込む。 そのため、画面の見た目と異なり、生成される画像はペースト前の古いプロンプトで実行されてしまう。
3. リロード時（F5 リフレッシュ）：
オートセーブ（1000ms デバウンス）完了前のリロード、あるいは `usePresetStoreSync.ts` によるサーバーDBフェッチ時に、 古いチップ状態および未保存のサーバープリセットでストアが再初期化される。

3. 具体的な改修設計方針とトレードオフの比較

本不具合を解消するためのアプローチとして、2つの設計案を比較・検討した。

比較項目	検討案 A（親レイアウト側の一括同期）【推奨・採用】	検討案 B（TagChipEditor 内部の通知ガード緩和）
実装概要	PromptSectionClipboardMenu の onChangeValue 実行時、親コンポーネント（DesktopLayout / MobileLayout）側で setParamValues と同時に parseToChips(val) を実行し setPromptChips も呼び出す。	TagChipEditor の value 外部更新検知分岐内で、isExternalUpdateRef によるブロックを回避し、onChipsChange(newChips) を直接呼び出す。
メリット	• エディタ内部の複雑なライフサイクル（D&D;・Undo/Redo履歴）に手を加えず安全。 • データフローの上流で文字列とチップの一貫性が100%保証される。 • 修正範囲が局所的で回帰バグのリスクが極めて低い。	• エディタを利用する全ての親画面で自動的に同期恩恵を受けられる。
デメリット / リスク	• DesktopLayout と MobileLayout の2箇所にハンドラ修正を適用する必要がある。	• 親が onChipsChange を受けて chipsProp を更新した際、TagChipEditor に戻り再同期ループ（無限レンダリング）を引き起こすリスクが高い。 • 単体テスト80件超への影響範囲が大きい。
総合判定	◎ 採用（最も堅牢かつ低リスク）	△ 不採用（循環更新リスクあり）

4. 具体的な実装コード差分 (Before / After)

推奨案（検討案 A）に基づき、`DesktopLayout.tsx` および `MobileLayout.tsx` において、クリップボード操作からの文字列反映ハンドラを拡張する。

4.1 DesktopLayout.tsx の改修

```
//  frontend/src/pages/GeneratePage/components/DesktopLayout.tsx
//  ──── 既存コード (Before) ────
    role={m.role!}
    currentValue={paramValues[m.role!] ?? ''}
    onChangeValue={(val) => setParamValues({ ...paramValues, [m.role!]: val })}
    onOpenSaveAsTagSet={() => setSaveAsTagSetRole(m.role!)}
/>

//  ──── 改修コード (After) ────
    role={m.role!}
    currentValue={paramValues[m.role!] ?? ''}
    onChangeValue={(val) => {
        setParamValues({ ...paramValues, [m.role!]: val });
        //  チップ配列も同時にパースしてストアへ即時反映
        handleChipsChange(m.role!, parseToChips(val));
    }}
    onOpenSaveAsTagSet={() => setSaveAsTagSetRole(m.role!)}
/>
```

4.2 MobileLayout.tsx の改修

```
//  frontend/src/pages/GeneratePage/components/MobileLayout.tsx
//  ──── 改修コード (After) ────
    role={activePromptTabMapping.role!}
    currentValue={paramValues[activePromptTabMapping.role!] ?? ''}
    onChangeValue={(val) => {
        setParamValues({ ...paramValues, [activePromptTabMapping.role!]: val });
        //  チップ配列も同時にパースしてストアへ即時反映
        handleChipsChange(activePromptTabMapping.role!, parseToChips(val));
    }}
    onOpenSaveAsTagSet={() => setSaveAsTagSetRole(activePromptTabMapping.role!)}
/>
```

4.3 必要となる import 追加

`DesktopLayout.tsx` および `MobileLayout.tsx` の冒頭に、`useChipParser` から `parseToChips` のインポートを追加する：

```
import { parseToChips } from '../../../components/TagChipEditor/useChipParser'
```

5. 検証計画と受入基準 (Verification Plan)

AC ID	受入基準の内容	検証方法
AC-1	クリップボードメニューから「上書き」で貼り付けた際、paramValues と同時に promptChips が即座に最新タグ一覧へ同期更新されること。	単体テスト (Vitest)
AC-2	テキストモードを経由することなく、上書きペースト直後に他セット（Set 2 等）へ切り替えて戻っても、ペースト内容が維持されていること。	シナリオテスト & 手動検証
AC-3	モバイル表示においてプロンプトタブ（Positive / Negative 等）を切り替えて戻っても、ペースト内容が巻き戻らないこと。	単体テスト (MobileLayout.test)
AC-4	上書きペースト後、直ちに画像生成を実行した際、ComfyUI に投入されるプロンプトがペースト後の最新タグと一致すること。	useGenerateAction テスト
AC-5	既存の「末尾に追加」モード、タグドラッグ＆ドロップ、手動テキスト編集、Undo/Redo の挙動が一切破壊されないこと。	全件テストスイート (npm test)

5.1 実行検証コマンド

1. 型検査: cd frontend; npm run typecheck (tsc -b --noEmit)
2. 単体テスト: npm test -- src/pages/GeneratePage/components/__tests__/PromptSectionClipboardMenu.test.tsx
3. 全件テスト: npm test -- --run (全84テストファイル完全パスを確認)